AddressSanitizer Stage 1 Replication

Local environment, correctness checks, substitute benchmarks and a controlled extension

25 October 2026 | Stage report, not a final submission

周潇洲 1240025753 | 张瀚元 1240025908
章殷远 1230022374 | 章越翔 1230019110

Stage verdict

Overall verdict: the detection behavior and overhead-measurement method were partially reproduced, while the original SPEC aggregate remains unestablished. The environment and a traceable experiment are complete. The tested faults were detected and valid/repaired controls passed. Performance uses a substitute Olden subset.

Table 1. Stage evidence summary

Item	Recorded evidence
Correctness and controls	45 / 45 passed; 5 runs per case
Performance	5 workloads x 3 configurations x 5 runs = 75 trials
Extension	2 workloads x 3 redzone settings x 5 runs = 30 trials
Mean time ratio	Modern: 7.663x; paper-like: 2.243x

The aggregate is an unweighted arithmetic mean of workload-level paired ratio means, not a SPEC score. Tables and charts are generated from local logs, not estimated results.

Evidence: records\stage1\20261025T072502Z-06ebb2

1. Environment and scope

The existing Ubuntu installation was reused to stay closer to the paper's 64-bit Linux evaluation. WSL scheduling and virtualization remain deviations. PERIOD files were archived and its two images removed; the Ubuntu virtual disk was preserved and not compacted.

Table 2. Recorded local environment

Item	This study
Host	Windows 11, Intel Core i5-13500HX (14 cores / 20 threads), about 15.7 GiB usable RAM
Guest	Ubuntu 22.04.5 LTS; WSL2 kernel 6.18.54.1; 6 visible CPUs; 6 GiB memory limit
Tools	Clang/LLVM 14.0.0 (Ubuntu 1:14.0.0-1ubuntu1.1); CMake 3.22.1; Ninja 1.10.1; lit 14.0.6
Benchmark commit	d52a93a891cde7a76aca63f7a201c5a5fe459f41
Workloads	bisort, mst, perimeter, treeadd, tsp; unmodified upstream Olden sources; reference input size
Execution	Linux ext4 work directory; one Linux logical CPU pinned; sequential measured processes; randomized block order, seed 340

Deviations from the paper

Table 3. Deviations from the paper

Item	Paper	Ours; reason
Hardware/OS	Dual Xeon E5620, 24 GB, Linux	Laptop / WSL2; available local machine
Tool/workload	2012 implementation; SPEC CPU2006	Clang 14; Olden; no institutional SPEC license; instructor allowed substitution
Optimization	clang -O2	-O2, frame pointers, non-PIE for both variants; fixed executable layout and readable traces
Memory metric	VmPeak at termination	Peak RSS from GNU time; measures resident memory and is not directly comparable
Coverage	Full SPEC suite, writes-only mode	Five Olden programs; full read/write ASan only; bounded first stage

Both variants share optimization and layout settings. The instrumented variant adds ASan, with use-after-return and use-after-scope instrumentation disabled in the performance build to keep the tested scope closer to 2012.

Build compatibility: direct Olden subtree configuration needs a generated lit.local.cfg with traditional_output=True; upstream C sources remain unchanged. Measured stdout is captured through a pipe and live timer output goes to Linux ext4 before being copied to the repository, reducing DrvFS logging effects.

2. Mechanism and correctness

Clang inserts checks around memory accesses. Shadow memory records whether the target bytes are accessible, while poisoned redzones surround objects. Quarantine delays reuse of freed allocations. On a detected fault the runtime reports the error category and call stack, then exits [1,2].

This detects memory faults exercised by the test; it does not prove their absence. Each fault is a separate process because ASan normally exits after the first detected error.

Table 4. Fault detection and clean controls

Case	Expected	Passed
heap	heap-buffer-overflow	5/5
stack	stack-buffer-overflow	5/5
global	global-buffer-overflow	5/5
uaf	heap-use-after-free	5/5
double_free	attempting double-free	5/5
control-heap	clean exit	5/5
control-stack	clean exit	5/5
control-global	clean exit	5/5
control-fixed	clean exit	5/5

A fault passes only if the exit is nonzero, the expected category is present, and the trace contains the source filename and line. Index 7 is valid and index 8 is out of bounds. The fixed control bounds the index before reading.

These trials support detection of the five constructed cases. Clean finite controls do not establish universal absence of false positives. The Chromium deployment and discovery of hundreds of previously unknown defects were not reproduced.

Traceability: correctness.json -> correctness/<case>-<run>.log; examples/memory_cases.c; source-snapshot/.

3. Runtime experiment and results

Each workload/configuration has one excluded warm-up and five measured processes. Configurations and workloads are shuffled within blocks using seed 340. Outputs must match across all trials, and lit validates against upstream references.

Elapsed time uses a monotonic high-resolution timer and includes startup/exit. Short workloads may be sensitive to launch cost. Ratios are computed within the same repetition, then summarized as mean +/- sample standard deviation (SD). SD is not an error bound.

Table 5 and Figure 1 summarize paired runtime overhead; per-configuration elapsed times are in the raw records.

Table 5. Paired runtime ratios and sample SD

Workload	Native s	Modern ASan / native	Paper-like / native
bisort	0.253 +/- 0.050	2.447 +/- 0.496x	1.770 +/- 0.267x
mst	0.030 +/- 0.005	1.884 +/- 0.251x	1.841 +/- 0.077x
perimeter	0.092 +/- 0.016	6.000 +/- 0.783x	1.913 +/- 0.111x
treeadd	0.106 +/- 0.019	24.728 +/- 5.356x	3.830 +/- 0.807x
tsp	0.266 +/- 0.027	3.260 +/- 0.288x	1.862 +/- 0.134x

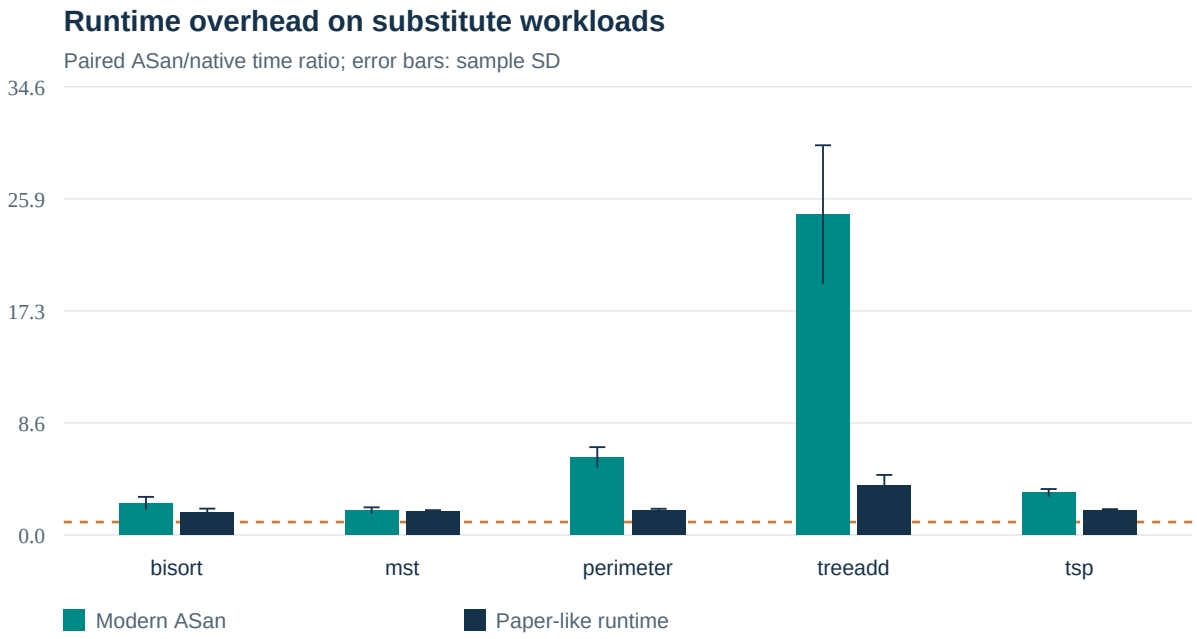

Figure 1. Paired time ratios; bars show means and whiskers show sample SD. Native reference = 1.0.

The modern profile retains allocator-context, redzone and quarantine defaults while explicitly disabling leak detection. The paper-like profile sets redzone=max_redzone=32, quarantine=0 and malloc_context_size=0. Neither profile is the 2012 binary.

Traceability: measurements.csv / .json -> performance/<workload>/<configuration>-<run>.log and .time. Warm-ups are recorded separately.

4. Memory, binary size and parameters

Peak RSS is the high-water mark of resident memory. It differs from the paper's Table 1 VmPeak metric. Modern 64-bit ASan maps a large virtual shadow region, so virtual mappings must not be interpreted as physical RAM consumption [2].

Table 6. Peak resident memory and executable size

Workload	Native RSS MiB	Modern RSS MiB	Paper-like RSS MiB	No-debug file ratio
bisort	17.83 +/- 0.03	131.61 +/- 0.01	40.46 +/- 0.05	85.07x
mst	26.36 +/- 0.00	36.48 +/- 0.02	36.24 +/- 0.01	85.45x
perimeter	86.86 +/- 0.00	282.32 +/- 0.05	124.46 +/- 0.05	86.57x
treeadd	129.48 +/- 0.00	1088.57 +/- 0.04	292.56 +/- 0.06	86.96x
tsp	66.06 +/- 0.06	316.81 +/- 0.04	112.78 +/- 0.03	68.74x

File ratios use complete executables after stripping debug information. Each variant was built once, so no repetition SD applies. Runtime and other fixed costs can dominate small executables. Section-size logs from llvm-size are also retained.

Why retain a separate correctness profile?

Smaller redzones reduce protected range; disabling quarantine can weaken use-after-free detection; removing allocation stack contexts reduces diagnostic detail. The paper's performance configuration is a tradeoff, not the strongest detection configuration. Correctness trials use the modern diagnostic profile.

Inputs used

Table 7. Upstream reference input arguments

Workload	Arguments from generated upstream .test
bisort	700000
mst	1000
perimeter	10
treeadd	22
tsp	1024000

Traceability: binary-sizes.json; sizes/*-sections.log; workloads.json; build/*-reference-validation.log; environment/runtime-files.log.

5. Extension: minimum heap redzone

Question: how does minimum heap redzone size (16, 32, 128 bytes) affect time and resident memory on the same inputs? Only redzone changes. max_redzone remains 2048, quarantine stays zero, and malloc_context_size stays zero. The factor is the minimum, because the modern allocator adapts actual redzone sizes.

See Table 8 and Figure 2. The two main runtime profiles change several settings together and do not isolate one causal factor.

Table 8. One-factor minimum heap redzone results

Workload	Minimum redzone	Time, s (mean +/- SD)	Peak RSS, MiB (mean +/- SD)
bisort	16	0.420 +/- 0.076	31.46 +/- 0.03
bisort	32	0.428 +/- 0.021	40.46 +/- 0.03
bisort	128	0.607 +/- 0.127	94.49 +/- 0.01
mst	16	0.054 +/- 0.010	36.47 +/- 0.03
mst	32	0.050 +/- 0.001	36.48 +/- 0.01
mst	128	0.057 +/- 0.014	36.46 +/- 0.05

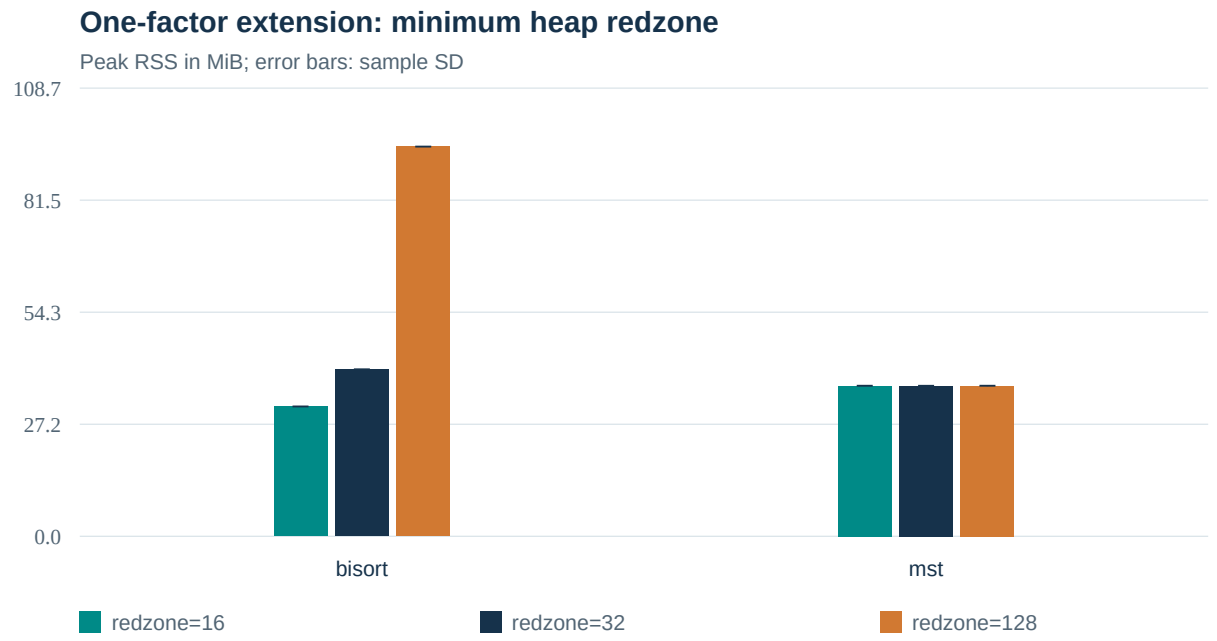

Figure 2. Measured peak RSS under one changed runtime parameter; means and sample SD, n=5.

Mean RSS change from 16 to 128 bytes: bisort: 3.00x (+200.4%); mst: 1.00x (-0.0%).

This tests a local resource-overhead claim about redzone tuning. It does not estimate real-world bug detection probability or establish the same trend for all programs.

6. Paper comparison and limitations

Table 9. Paper claims versus stage evidence

Claim	Paper	Ours	Verdict
Fault categories (Sec. 3)	Heap/stack/global bounds; UAF	5 constructed faults detected, valid controls clean	Supported only for tested cases
Runtime (Fig. 2)	1.73x average on SPEC CPU2006	7.663x modern; 2.243x paper-like on Olden	Original numeric claim inconclusive: different workloads, compiler, host
Memory (Table 1)	3.37x aggregate VmPeak	Peak RSS measured separately	Inconclusive: different metric and workload
Tuning (Sec. 4.3)	Redzones affect resources	Controlled redzone experiment above	Evidence restricted to two inputs

Descriptive only: the paper-like unweighted time ratio differs numerically from 1.73 by +0.513x (+29.7% relative to 1.73). Different workloads make this unsuitable as a same-setting replication error or evidence of a hardware/compiler cause.

Controlled checks include identical compiler/source/input/optimization/affinity, upstream output validation, a modern-versus-paper-like profile comparison, and one-factor redzone re-runs. Windows background work, temperature/power policy and WSL virtual-to-P/E-core mapping remain uncontrolled. Five Olden programs have limited representativeness.

Next steps and course requirements

Next, add diverse C/C++ applications and input sizes, and repeat short programs under quieter conditions. Original SPEC numbers, writes-only instrumentation and Chromium bug discovery remain outside this stage. The final English report must retain statistics, deviations and traceable logs [4].

Group repository: github.com/zxz-codes/CS340-AddressSanitizer-Replication. Four members are registered; individual roles are deferred at the group's request. Each member must still make substantive commits from their own account, with ID and full name in messages. Weekly checks are Sunday 23:59; tag v1.0 only for the final release. This stage artifact is not a completed final submission.

Contributions and AI use

The group supplied the paper choice, course materials and instructor response allowing benchmark substitution. AI assisted with archiving, setup, scripts, educational cases, experiment execution, log analysis and this report. Measurements are from recorded runs. Member-specific design, review, analysis and presentation contributions must be confirmed by the four students.

References

[1] Serebryany et al. AddressSanitizer: A Fast Address Sanity Checker. USENIX ATC 2012, Secs. 3-4, Fig. 2, Tables 1-2. https://www.usenix.org/system/files/conference/atc12/atc12-final39.pdf

[2] Clang AddressSanitizer documentation. https://clang.llvm.org/docs/AddressSanitizer.html (accessed 25 Oct 2026).

[3] LLVM test-suite guide and repository, pinned commit above. https://llvm.org/docs/TestSuiteGuide.html ; https://github.com/llvm/llvm-test-suite

[4] CS340 Research Paper Replication, Academic Year 2026-2027, supplied course handout; instructor reply supplied as screenshot permits an open-source benchmark substitute.